为什么需要 LangChain

- 来源: https://class.imooc.com/lesson/5282#mid=105026
- 阶段: 阶段四: 【框架应用】高效AI开发
- 周: 第8周 LangChain框架入门
- 模块: LangChain核心概念与应用
- 课节: 图文 1-1 为什么需要 LangChain

1-1 为什么需要 LangChain

为什么需要 LangChain

> [!TIP] 阅读指南 在前面的章节中，我们学会了调用 API、设计 Prompt、构建 RAG 系统、使用 MCP 调用工具。但每次开发 AI 应用都要写很多重复代码——管理对话历史、解析 Function Calling、处理错误重试。这就像每次做饭都要从磨刀、洗菜、生火开始，太累了。

重复造轮子的痛苦

还是用之前的案例：开发一个游戏攻略助手。

需求

- 用户问问题
- 检索知识库找答案
- 如果需要实时数据(如最新版本信息),调用游戏 API
- 整合结果后返回

传统写法

```
# 1. 管理对话历史
messages = []

# 2. 调用 RAG 检索
def search_knowledge(query):
    embedding = get_embedding(query)
    results = vector_db.search(embedding)
    return results

# 3. 判断是否需要调用 API
def need_api_call(query, rag_results):
    prompt = f"根据检索结果判断是否需要调用游戏 API 获取实时数据..."
    response = openai.chat.completions.create(...)
    return "需要调用" in response

# 4. 调用游戏 API
def call_game_api(query):
    # 调用游戏数据 API
    ...

# 5. 组装完整流程
def answer_question(user_query):
    # 检索知识库
    rag_results = search_knowledge(user_query)

    # 判断是否需要调用 API
    if need_api_call(user_query, rag_results):
        api_data = call_game_api(user_query)
        context = rag_results + api_data
    else:
        context = rag_results

    # 生成回答
    messages.append({"role": "user", "content": user_query})
    response = openai.chat.completions.create(
        model="qwen3.6-plus"
        messages=[
            {"role": "system", "content": f"参考资料:\n{context}"},
            *messages
        ]
    )

    answer = response.choices[0].message.content
```

```
    messages.append({"role": "assistant", "content": answer})

    return answer
```

这段代码有什么问题?

- 重复代码多 - 每个项目都要写类似的对话管理、工具调用逻辑

重复代码多

- 不易维护 - 逻辑散落在各处，改起来麻烦

不易维护

- 缺少抽象 - 没有统一的接口，复用困难

缺少抽象

- 容易出错 - 手动管理状态，容易漏掉边界情况

容易出错

LangChain 的解决方案

LangChain 提供了标准化的组件，把常见的 AI 应用模式封装成可复用的积木。

用 LangChain 改写:

```
from langchain.chat_models import ChatOpenAI
from langchain.memory import ConversationBufferMemory
from langchain.tools import Tool
from langchain.agents import initialize_agent

# 1. 初始化组件
llm = ChatOpenAI(model="gpt-5")
memory = ConversationBufferMemory()

# 2. 定义工具
game_api_tool = Tool(
    name="game_api",
    func=call_game_api,
    description="获取游戏实时数据,如角色属性、版本信息等"
)

# 3. 创建 Agent（智能代理，能自主决定何时调用工具）
# Agent 的详细原理和实现，我们会在第9章详细讲解
agent = initialize_agent(
    tools=[game_api_tool],
    llm=llm,
    memory=memory,
    agent="conversational-react-description"
)

# 4. 使用
answer = agent.run("原神中雷电将军的命座效果是什么?")
```

- 对话管理 - ConversationBufferMemory 自动处理

对话管理

- 工具调用 - Agent（智能代理）自动决定何时调用游戏 API

工具调用

- 流程编排 - 框架自动处理检索→判断→API调用→生成的流程

流程编排

- 错误处理 - 内置重试和降级机制

错误处理

> 关于 Agent Agent（智能代理）是能够自主决策的 AI 系统，也是目前AI领域最火的前沿概念和技术。在LangChain中，Agent会根据用户问题，自动判断需要调用哪些工具、按什么顺序调用。关于 Agent 的详细原理和实现，我们会在第9章详细讲解。

LangChain 就像乐高积木

LangChain 的核心思想:把 AI 应用拆解成标准化的组件，像搭积木一样组装。

核心组件:

- LLMs - 大模型接口(OpenAI、Qwen、本地模型等)

LLMs

- Prompts - Prompt 模板管理

Prompts

- Chains - 把多个步骤串联起来

Chains

- Memory - 对话历史管理
- Tools - 外部工具调用

Tools

- Agents - 智能代理，让 AI 自主决策调用哪些工具（后续章节详述）

Agents

传统开发就像每次都手工制作零件，然后组装成产品。LangChain 提供了标准化的乐高积木，你只需要:

- 选择需要的积木(组件)
- 按照说明书组装(配置)
- 运行(调用)

■ 学点英语

中文	English	音标	说明
链式框架	LangChain	/læŋ tʃeɪn/	将 AI 应用拆解成标准化组件的集成框架
样板代码	Boilerplate Code	/ˈbɔɪlərpleɪt koʊd/	反复编写的基础性重复代码
编排	Orchestration	/ˌɔːrkɪˈstreɪʃn/	协调多个组件按流程协同工作
组件	Component	/kəmˈpoʊnənt/	可复用的模块化功能单元

下一节预告

我们目前只是通过一个案例大致感受了 LangChain 的作用，却还没有系统梳理这个框架本身：到底有哪些核心组件，它们之间如何协同工作，又该如何在真实项目中按需选择。接下来，我们会从概念层面把 LangChain 拆开来看，理清它的基本积木和连接方式，为后面搭建第一个完整应用打下清晰的认知基础。